

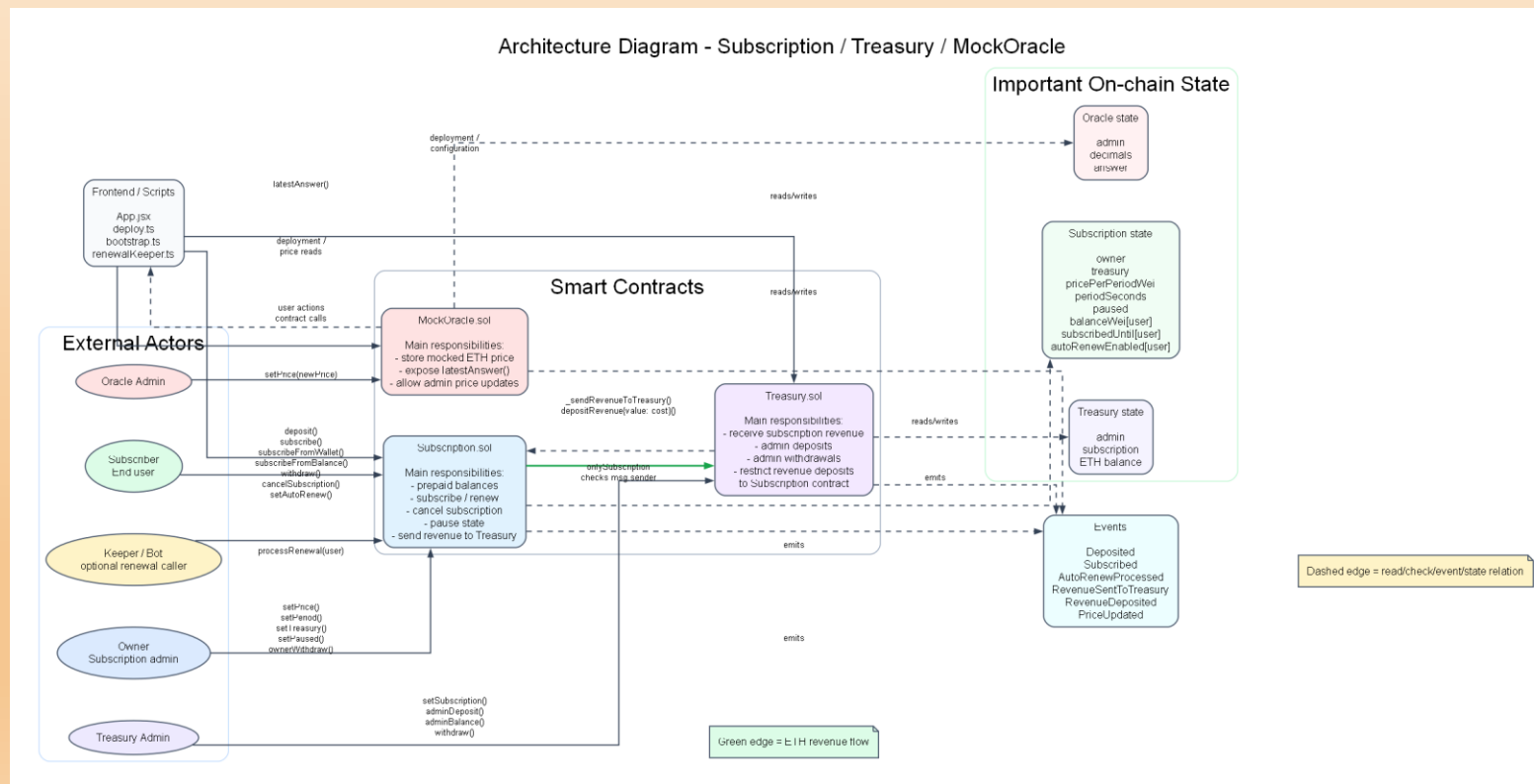
Testing a Blockchain Application Using Hardhat

T8 - Testarea unei rețele Blockchain

Focusul proiectului: testare, nu aplicatia in sine

Stack: Hardhat, Mocha, Chai, ethers, TypeScript

Directii urmarite: unit, integration, security, performance



Project Goal

De ce proiectul este relevant pentru Testarea Sistemelor Software

Application under test

Subscription system cu 3 contracte: Subscription, Treasury, MockOracle

Obiectiv Principal

Sa validam comportamentul contractelor si interactiunea lor pe o retea Hardhat locala

Why testing matters here

Aplicatiile blockchain manipuleaza fonduri si stare permanenta; o eroare logica este costisitoare

Nu am urmarit doar functii izolate, ci fluxurile esentiale ale sistemului

Exemple: deposit, subscribe, renew, transfer catre Treasury, validari de roluri

Abordarea a fost ghidata de cerinta T8: unit + integration + security + performance

Why Hardhat and This Stack

Alegerea tool-urilor de testare

Hardhat: retea blockchain locala, scripting, debugging si testare automata

Mocha + Chai: descriere clara a scenariilor si asertiilor

ethers: interactiune directa cu contractele si conturile din retea

TypeScript: teste mai lizibile si mai usor de organizat

Comparativ cu Foundry / Truffle, Hardhat s-a potrivit cel mai bine cu stilul proiectului

Component	Version
Node.js	v22.17.0
Hardhat	^2.22.3
Solidity	0.8.26
Mocha	^11.7.5
Chai	^4.3.10
ethers	^6.16.0

Test Suite Structure

Cum este organizata suita principala de teste

File	Main focus	Tests
test/Subscription.ts	subscription logic, state transitions, pause behavior	8
test/TreasuryAndOracle.ts	permissions, treasury flow, oracle restrictions	4
test/Performance.ts	gas consumption and comparative checks	4

Functional testing

Validarea comportamentului:
deposit, subscribe, renew, inputuri
valide si invalide

Integration testing

Flux complet intre Subscription si
Treasury, inclusiv trimiterea
fondurilor

Security testing

Restrictii de roluri, invalid input,
paused state, revenue restrictions

Performance testing

Masurarea costului de gas pentru
operatiile importante ale sistemului

Total in suita principala: 16 teste, toate trecute cu succes

Key implementation techniques: loadFixture, time.increase, revertedWithCustomError, to.emit(...).withArgs(...)

Functional Testing Strategy

Equivalence classes + boundary value analysis

Am grupat comportamentele in clase de echivalenta pentru: deposit amount, payment source, renewal state si caller permissions

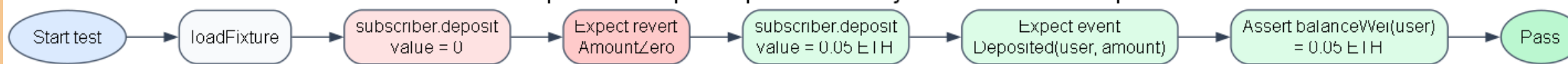
Am tratat explicit frontierele importante: 0 ETH, exact subscription price, insufficient funds, 0 periods, expirare vs. neexpirare

Testele verifica atat cazuri valide, cat si revert-uri cu custom errors

Exemple reprezentative: deposit(0), subscribe with insufficient funds, renewal before expiry, unauthorized admin call

Representative functional cases from README:
C1 deposit with 0 ETH -> AmountZero
C6 prepaid balance + wallet combined correctly
C10 renewal after expiry without funds -> not processed

02 - Subscription: accepts deposits and rejects zero-value deposits



Integration and Security Testing

Validam nu doar functii, ci si colaborarea si protectia sistemului

Integration: subscribe actualizeaza starea si trimite veniturile catre Treasury

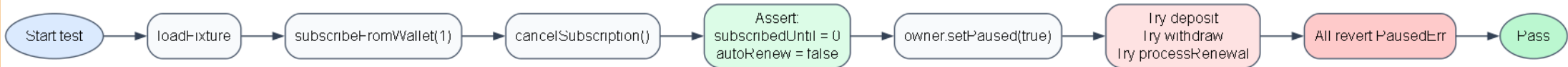
Security: doar owner/admin/subscription au acces la operatiile sensibile

Se verifica zero-value operations, unauthorized calls si paused state

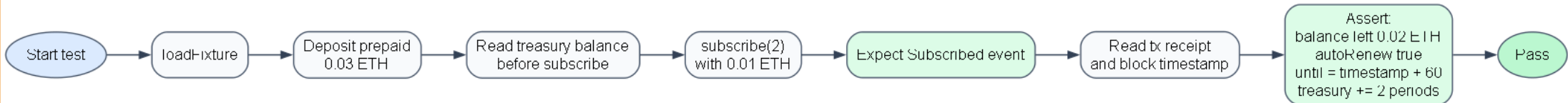
Scopul: contractele sa fie corecte si rezistente la utilizare gresita

Example security assertion:
unauthorized Treasury depositRevenue call ->
revertedWithCustomError(NotSubscription)

07 - Subscription: cancel and paused behavior



03 - Subscription: prepaid balance + wallet value



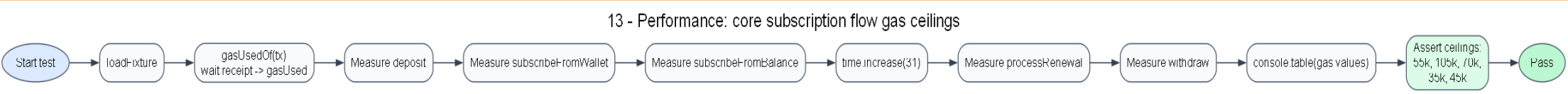
Performance Testing Through Gas

In blockchain, costul executiei este o parte a calitatii

Operation	Gas used
deposit	47360
subscribeFromWallet	98429
subscribeFromBalance	61429
processRenewal	28875
withdraw	38075
treasury.adminDeposit	22858
treasury.withdraw	33571

subscribeFromWallet este cel mai scump flux testat
renewal este mai ieftin decat o prima abonare din wallet

comparatia exact payment vs leftover credit sustine designul functional



Coverage Results

Ce parte din logica contractelor este exercitata de teste

Statement coverage: 83.87% (78 / 93)

Branch coverage: 61.82% (68 / 110)

Function coverage: 70.97% (22 / 31)

Line coverage: 85.04% (108 / 127)

Contract	Statements	Branches	Functions	Lines
MockOracle.sol	100%	83.33%	100%	100%
Subscriptions.sol	80.82%	56.58%	60%	81.82%
Treasury.sol	93.33%	71.43%	87.5%	94.74%

/									
83.87% Statements 78/93 61.82% Branches 68/110 70.97% Functions 22/31 85.04% Lines 108/127									
File	Statements	Branches	Functions	Lines					
contracts/		83.87%	78/93	61.82%	68/110	70.97%	22/31	85.04%	108/127

all files contracts/									
83.87% Statements 78/93 61.82% Branches 68/110 70.97% Functions 22/31 85.04% Lines 108/127									
File	Statements	Branches	Functions	Lines					
MockOracle.sol		100%	5/5	83.33%	5/6	100%	3/3	100%	9/9
Subscriptions.sol		80.82%	59/73	56.58%	43/76	60%	12/20	81.82%	81/99
Treasury.sol		93.33%	14/15	71.43%	20/28	87.5%	7/8	94.74%	18/19

AI-Assisted Testing

AI ca instrument complementar, nu ca inlocuitor

Criterion	Experiment 1	Experiment 2	Team suite
Workflow	ChatGPT ideas + Codex	Codex with repository context	Manual design
Tests passing	6/6	12/12	16/16
Scope	basic missing checks	broader missing checks	core project coverage
Role	starting point	more reusable extension	baseline reference

Experiment 1 a fost util pentru idei si verificari simple, dar a ramas mai restrans
Experiment 2 a fost clar mai bun dupa ce AI-ul a primit contextul real din repository
Concluzia: contextul imbunatateste semnificativ calitatea testelor generate

Concluzii

Ce demonstrem prin acest proiect

Am construit o suita de teste axata pe cerinta T8: unit, integration, security, performance

Am validat atat logica locala a contractelor, cat si fluxurile dintre ele

Am masurat costul de gas si am analizat coverage-ul, nu doar rezultatul pass/fail

AI-ul a fost util ca extensie, dar suita scrisa de echipa ramane referinta principala

Multumim!

```
PS C:\Users\USER\Desktop\Test_Sys\Project---Testarea-Sistemelor-Software> npm test
```

Performance

(index)	operation	gasUsed
0	'deposit'	'47360'
1	'subscribeFromWallet'	'98429'
2	'subscribeFromBalance'	'61429'
3	'processRenewal'	'28875'
4	'withdraw'	'38075'

- ✓ keeps the core subscription flow under reasonable gas ceilings (669ms)
- ✓ shows subscribing with leftover wallet credit costs more gas than an exact wallet payment
- ✓ shows renewal is cheaper than a first-time wallet subscription

(index)	operation	gasUsed
0	'treasury.adminDeposit'	'22858'
1	'treasury.withdraw'	'33571'

- ✓ keeps treasury admin operations under reasonable gas ceilings

Subscription

- ✓ stores deployment configuration (38ms)
- ✓ accepts deposits and rejects zero-value deposits
- ✓ subscribes by combining prepaid balance with wallet value and forwards revenue to the treasury
- ✓ keeps wallet leftovers as prepaid balance when subscribing directly from the wallet
- ✓ extends an active subscription when subscribing from prepaid balance
- ✓ processes auto-renewal only after expiry when prepaid funds are available
- ✓ cancels subscriptions and blocks state-changing actions while paused
- ✓ restricts admin functions to the owner

Treasury and MockOracle

- ✓ only allows the configured subscription contract to deposit revenue
- ✓ lets the admin fund and withdraw from the treasury
- ✓ restricts treasury admin functions to the admin account
- ✓ allows only the oracle admin to update the price and rejects invalid values

16 passing (882ms)